



# Robotics & Computer Vision

## Week 13: Obstacle Avoidance & Autonomous Navigation

Combining AI detection + navigation logic for Turtlebot 4 to autonomously reach a goal while handling obstacles in real time.

Lê Đình Phong, Ph.D

## Week 13 Learning Goals

What you'll be able to do after this week:



### Understand the Theory

Distinguish reactive navigation, potential field, VFH, and simple behavior strategies.



### Write the Code

Build the full pipeline: AI detects obstacles → compute avoidance direction → send movement commands to Turtlebot.



### Run a Real Demo

Robot navigates autonomously from A to B, stopping or steering around people and obstacles detected by AI.

## Week 13 in the RCV Program

### Bridge from Week 12

You already know pose estimation → simple path planning. This week adds **dynamic obstacle avoidance** on top.

### Core Focus

Combine AI obstacle detection with navigation logic (Vector Field Histogram or equivalent) so Turtlebot moves to a goal and reacts to people and objects detected by AI.

### Foundation for Week 14

Navigation + obstacle avoidance are the core components of the **final integration demo**. Get this right now, and Week 14 becomes a polish sprint.

 No full SLAM required — odometry from Week 12 is sufficient to estimate direction to goal.

## The Big Picture

What does a fully autonomous robot system actually need?



### Perception

Camera, LiDAR, encoder — understand the environment.



### Localization

Know where the robot is right now (covered in Week 12).



### Planning

Compute a path from A to B that avoids known obstacles.



### Control

Convert the planned path into actual wheel commands.

## Deliberative vs. Reactive Navigation

### Deliberative

- Requires a complete map
- Computes optimal path ( $A^*$ , Dijkstra)
- Slower, more compute-intensive
- Depends on map accuracy

### Reactive

- No full map needed
- Processes sensor data in real time
- Simple and fast to implement
- Prone to local minima traps

✓ Week 13 uses **reactive as the primary approach** — ideal for AI students: simple code, immediate results.

## Our Approach This Week

### **Reactive as Primary**

Well-suited for AI students  
— simple to code, results  
visible immediately without  
heavy map infrastructure.

### **Goal-Directed Overlay**

The robot knows its  
destination and steers toward  
it, while obstacle avoidance  
operates reactively on top.

### **No Full SLAM Required**

Odometry from Week 12 is  
sufficient to estimate the  
direction toward the goal at  
each control cycle.

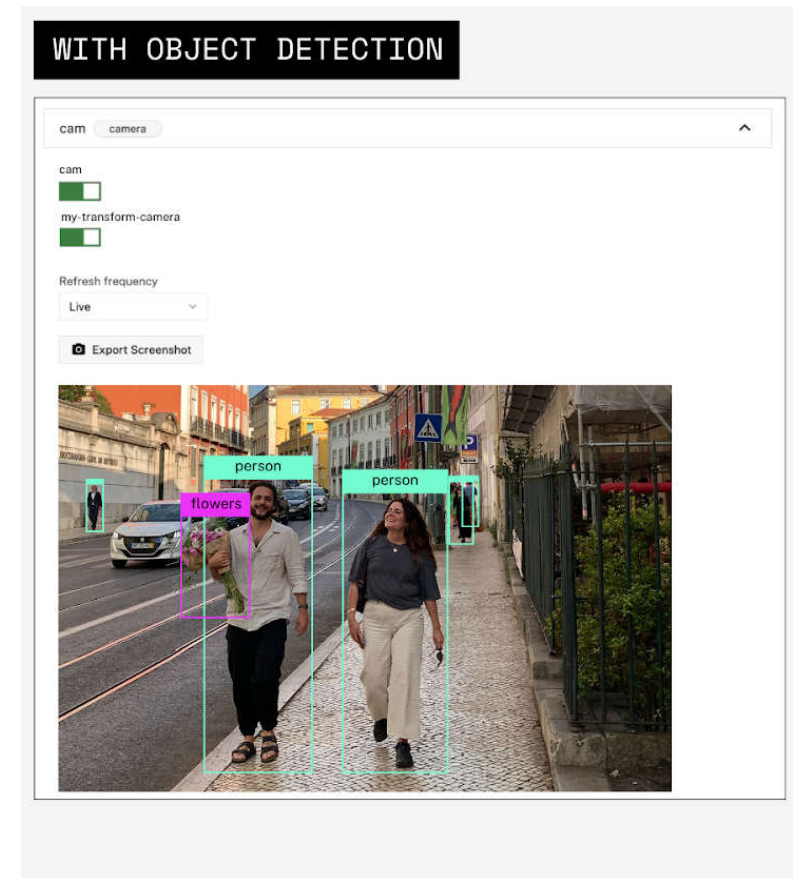
## AI Perception: Detecting Obstacles

### YOLO (from Week 4)

- Detects people, objects, vehicles in real time
- Runs on small GPU or CPU
- Returns bounding box with class + confidence

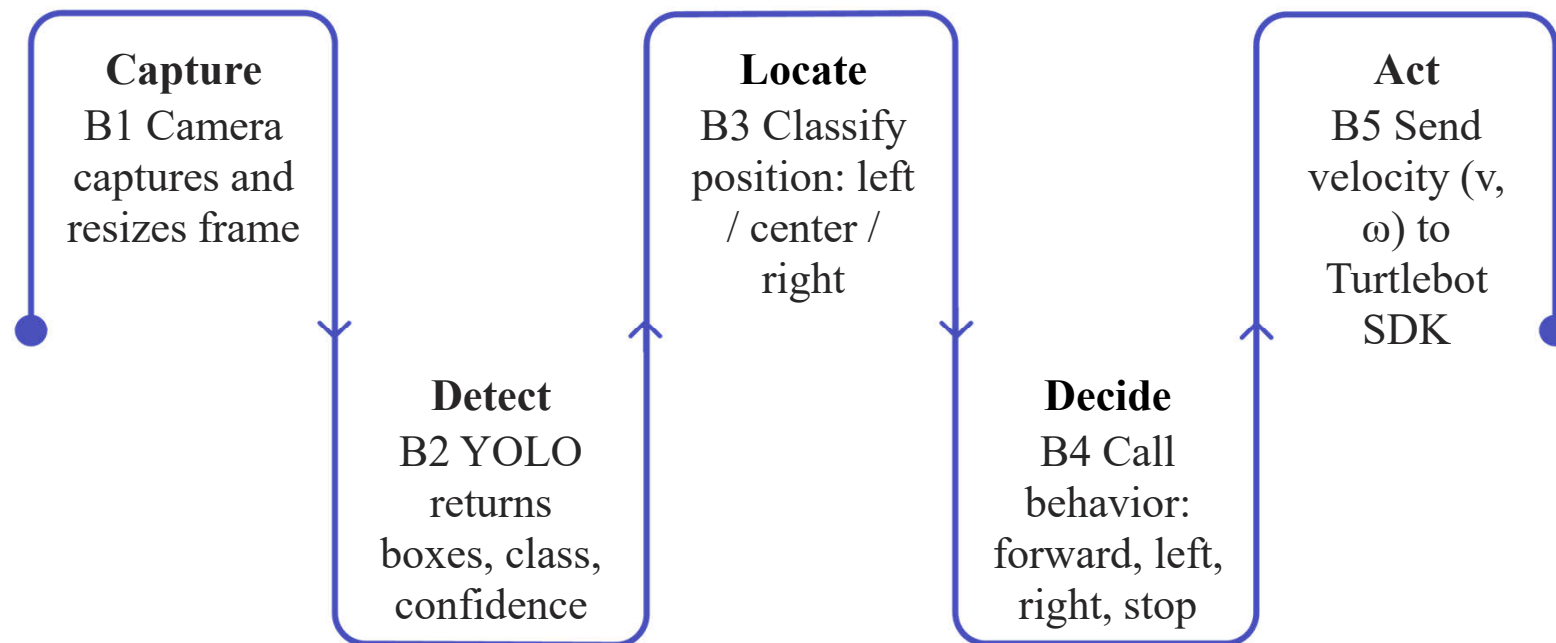
### From Bounding Box → Distance Estimate

- Larger box area → object is closer
- Combine with depth camera for precision
- Minimum: extract direction — left, center, or right



## From Pixel to Decision

The core pipeline: Camera frame  $\rightarrow$  Robot action



This pipeline runs inside a tight control loop, giving the robot continuous situational awareness from raw camera input to wheel commands.



## Zone-Based Obstacle Detection

Divide the image into three horizontal zones based on the detected bounding box center:

### ← Left Zone

Box center  $x < W/3$

Object on the left → **turn right**

### ↑ Center Zone

$W/3 \leq x < 2W/3$

Object ahead → **stop or rotate**

### → Right Zone

$x \geq 2W/3$

Object on the right → **turn left**

This approach requires no depth sensor and works well for indoor lab demos — fast, interpretable, and easy to debug.

## Code Skeleton: Zone Classification

Classifying which zone an obstacle belongs to, based on the bounding box center x-coordinate:

```
def get_zone(cx, img_w):  
    if cx < img_w / 3:  
        return "left"  
    elif cx < 2 * img_w / 3:  
        return "center"  
    else:  
        return "right"
```

- ⓘ This single function is the backbone of the entire zone-based obstacle avoidance system. Test it thoroughly with edge cases before connecting to the robot.

## Behavior-Based Control

Simple lookup table mapping situation  $\rightarrow$  robot action:

| Situation         | Behavior                | Command ( $v, \omega$ )                  |
|-------------------|-------------------------|------------------------------------------|
| No obstacle       | Move toward goal        | $v = 0.2, \omega = \omega_{\text{goal}}$ |
| Obstacle on left  | Turn right              | $v = 0.1, \omega = -0.5$                 |
| Obstacle on right | Turn left               | $v = 0.1, \omega = +0.5$                 |
| Obstacle center   | Rotate in place or stop | $v = 0, \omega = \pm 0.8$                |
| Near goal         | Stop                    | $v = 0, \omega = 0$                      |

## Potential Field — The Intuition

### Attractive Force from Goal

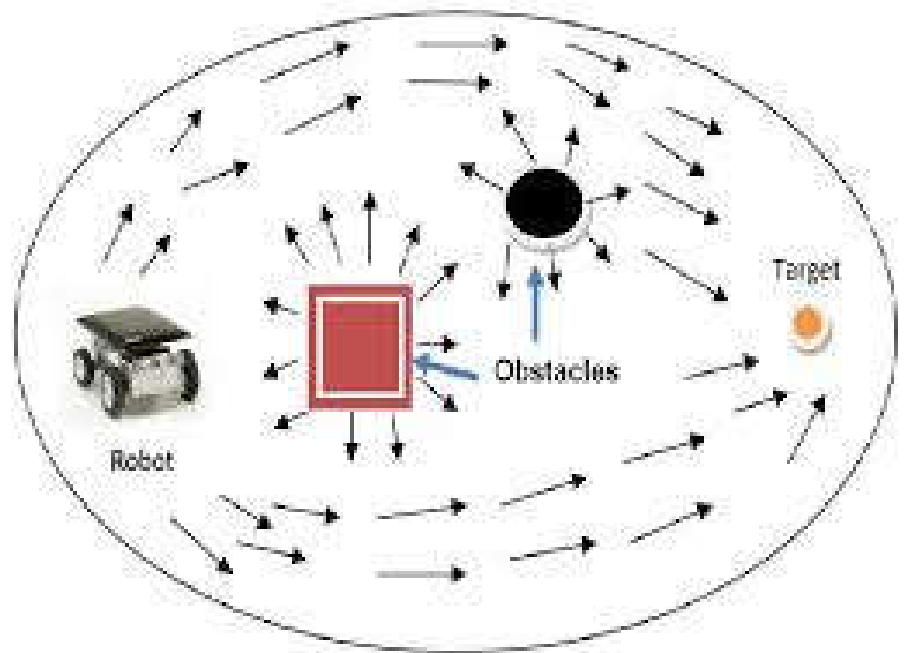
The robot is "pulled" toward the goal. Magnitude is proportional to distance.

$$F_{\text{att}} = K_{\text{att}} \times (\text{goal} - \text{pos})$$

### Repulsive Force from Obstacles

The robot is "pushed away" when too close to an obstacle. Force drops to zero beyond influence radius.

$$F_{\text{rep}} = K_{\text{rep}} / d^2 \times \text{avoidance\_dir}$$



## Potential Field: From Force to Command



Compute  $F_{\text{total}}$

Find  $\theta_{\text{desired}}$

Compute  $\omega$

This is one of the most intuitive ways to teach potential fields to AI students — it directly connects vector math to a P-controller, making the physics tangible.

## Limitations of Potential Field

### ⚠ Local Minima

Robot gets stuck at a point where  $F_{att}$  and  $F_{rep}$  cancel each other out — but it's not the goal.

### ⚠ Oscillation

Robot rocks back and forth when navigating through narrow corridors due to alternating repulsive forces.

### ⚠ Unreachable Goal

When an obstacle completely blocks the path to the goal, the robot may never arrive.

⚠ These are well-known limitations. VFH was designed specifically to address them.

## Vector Field Histogram (VFH)

### The Core Idea

- Divide  $360^\circ$  into angular sectors
- Count obstacle density in each sector (from LiDAR or depth)
- Find the most "open" sector closest to the goal direction
- Steer the robot toward that sector

### Advantages over Potential Field

- Much less prone to local minima
- Handles LiDAR scan data natively
- More intuitive geometric explanation
- Widely used in real-world robot navigation

## Simplified VFH for Teaching

A 3-sector version that's effective enough for the Week 13 demo:

### Left Sector

Count LiDAR/depth points in the left zone. High count = obstacle-heavy.

### Center Sector

Count points directly ahead. This is the most critical zone to monitor.

### Right Sector

Count points in the right zone. Compare all three to find the clearest path.

Select the sector with the **lowest density** that is closest to the goal direction — that becomes the next heading. This "teaching VFH" is effective for the Week 13 demo.



## Comparing the Three Approaches

| Method              | Advantages                    | Disadvantages           | Week 13?              |
|---------------------|-------------------------------|-------------------------|-----------------------|
| Zone-based (camera) | Simple, fast, no extra sensor | No distance estimate    | ✓ Yes                 |
| Potential Field     | Smooth, intuitive to explain  | Local minima risk       | ✓ Yes                 |
| VFH                 | Robust, production-ready      | Requires LiDAR or depth | ✓ If sensor available |

## Goal-Directed Navigation

How does the robot actually drive toward a destination?

### Computing the Heading

```
 $\alpha$  = atan2(dy, dx)      # angle to goal  
e_h =  $\alpha$  -  $\theta_{\text{robot}}$   # heading error  
 $\omega$  =  $K_p \times e_h$         # angular velocity  
 $v$  =  $v_{\text{max}} \times (1 - |e_h|/\pi)$  # slow when turning
```

Speed is reduced as heading error increases — the robot prioritizes alignment before acceleration.

### When an Obstacle Appears

The avoidance command **overrides** the goal-following command. Once the obstacle is clear, the robot automatically resumes steering toward the goal.

- ✓ This override-and-resume pattern is the heart of the Week 13 demo behavior.

## Subsumption Architecture

A layered behavior framework where lower layers override higher layers when triggered:

1

### Layer 2: Go to Goal

Normal navigation — active when path is clear.

2

### Layer 1: Avoid Obstacle

Overrides Layer 2 when obstacle detected in danger zone.

3

### Layer 0: Emergency Stop

Highest priority — halts robot if object is dangerously close. Cannot be overridden.

Higher layers are "subsumed" by lower layers when their activation conditions are met — a practical, widely used architecture for behavioral robotics.

## Behavior Priority in Code

```
def select_action(obstacle_zone, dist_to_goal):  
    if dist_to_goal < 0.2:          # arrived  
        return stop()  
    if obstacle_zone == "center":  
        return avoid_front()  
    if obstacle_zone == "left":  
        return turn_right()  
    if obstacle_zone == "right":  
        return turn_left()  
    return go_to_goal()             # clear path
```

- ① This function is evaluated every control cycle (~20 Hz). The order of conditions encodes the subsumption hierarchy directly in code.

## Turtlebot 4 SDK: Sending Commands

### Connecting with iRobot Create 3 SDK

```
from irobot_edu_sdk.backends.bluetooth import  
Bluetooth  
from irobot_edu_sdk.robots import Create3  
  
robot = Create3(Bluetooth())
```

### Basic Movement Commands

```
await robot.move(0.3)           # forward 0.3m  
await robot.turn_left(90)       # rotate 90°  
await robot.set_wheel_speeds(v_l, v_r)
```

Use `set_wheel_speeds` for continuous control — it gives fine-grained real-time velocity adjustment.

## The Continuous Control Loop

```
while not arrived:
    frame = camera.get_frame()
    boxes = yolo.detect(frame)
    zone = classify_zone(boxes, frame.width)
    pose = odometry.get_pose()
    v, w = select_action(zone, distance_to_goal(pose))
    robot.set_velocity(v, w)
    await asyncio.sleep(0.05)    # 20 Hz
```

Every 50ms the robot re-evaluates perception, planning, and control — all in one tight loop. Keeping this loop fast and non-blocking is critical for safe operation.

## Real-Time Challenge: AI Is Slower Than Control

### The Problem

YOLO inference takes **50–200ms** — far too slow to block a 50ms control loop. Waiting for AI means the robot freezes between detections.

### The Solution: Separate Threads

- AI runs in a background thread, continuously updating a shared `obstacle_zone` variable
- The control loop reads the latest value without waiting for inference
- Robot stays responsive even during heavy AI computation

## Threading Architecture: AI + Control

```
import threading

obstacle_state = {"zone": None}

def ai_worker():
    while True:
        frame = cam.read()
        boxes = yolo.detect(frame)
        obstacle_state["zone"] = classify_zone(boxes)

t = threading.Thread(target=ai_worker, daemon=True)
t.start()

# Control loop reads obstacle_state["zone"] each cycle
```

- ④ Using `daemon=True` ensures the AI thread stops automatically when the main program exits — no zombie threads left behind.



## Safety First: Emergency Stop



### Try / Finally Pattern

Guarantees the robot stops even if code crashes at any point during execution.



### Minimum Distance Check

If bounding box area exceeds a threshold (object very close), stop immediately regardless of other logic.



### Timeout Guard

If the robot hasn't made progress toward the goal after X seconds, stop and raise an error. Prevents endless wandering.

## Emergency Stop Pattern

```
try:
    await run_robot_loop()
except Exception as e:
    print("Error:", e)
finally:
    await robot.set_wheel_speeds(0, 0) # always stop
    robot.disconnect()
```

- ⊗ Never run a physical robot without this pattern. Hardware failures are unpredictable — always guarantee a clean shutdown path in your code.

## Detecting Goal Arrival

### Using Odometry

Compute Euclidean distance from current pose to goal. If  $d < \text{threshold}$  → arrived.

```
d = sqrt((gx-x)2 + (gy-y)2)
if d < 0.25:
    arrived = True
```

### Using Visual Markers

- **ArUco marker** placed at goal — camera detects ID → stop
- **YOLO class** "goal\_marker" detected with high confidence → stop

✓ ArUco is more robust for demo scenarios — it gives a clear, unambiguous arrival signal.

## Estimating Distance from Bounding Box

### Simple Formula (Pinhole Model)

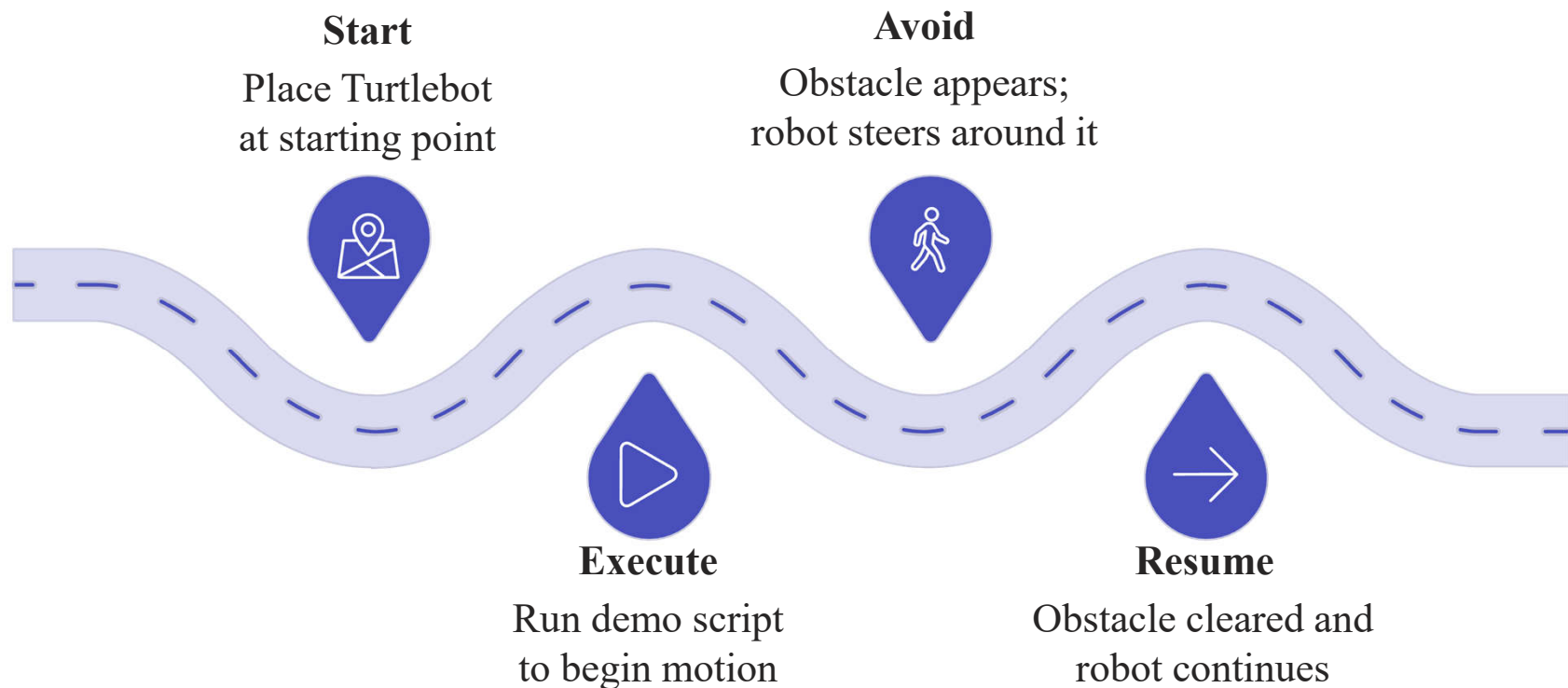
$$\text{dist} \approx (f \times h_{\text{real}}) / h_{\text{box}}$$

Where  $f$  = focal length,  $h_{\text{real}}$  = real object height,  $h_{\text{box}}$  = height of bounding box in pixels.

### Practical Calibration

- Place object at 0.5m, 1.0m, 1.5m and log  $h_{\text{box}}$  for each
- Fit a simple regression:  $\text{dist} = a / h_{\text{box}} + b$
- Calibrate once per camera setup — values differ between cameras

## Standard Demo Scenario: Week 13



This is the benchmark scenario every team should be able to complete before the Week 14 integration demo.

## Key Parameters to Tune

| Parameter                   | Effect                         | Starting Value  |
|-----------------------------|--------------------------------|-----------------|
| $v_{\text{forward}}$        | Speed toward goal              | 0.15 – 0.25 m/s |
| $\omega_{\text{turn}}$      | Rotation speed when avoiding   | 0.5 – 0.8 rad/s |
| $K_{\text{goal}}$           | Goal heading gain              | 1.0 – 2.0       |
| $\text{danger\_box\_ratio}$ | Box/frame threshold for danger | 0.15 – 0.25     |
| $\text{arrived\_threshold}$ | Distance considered "at goal"  | 0.2 – 0.35 m    |

Start with the conservative lower bound for each parameter and increase gradually during testing.

LAB 13.1

# Lab 1: Zone-Based Obstacle Avoidance

01

## Run YOLO on Webcam

Detect "person" or any object class in real time from your webcam feed.

03

## Print the Action

Log the corresponding behavior (no robot connection yet). Verify correctness with at least 3 different scenarios.

02

## Call `get_zone(cx, W)`

For each detection, classify the obstacle into left, center, or right zone.

04

## Visualize

Draw bounding boxes and zone labels directly on the camera image using OpenCV.

LAB 13.2

# Lab 2: Goal-Following Without Obstacles

01

## Use Odometry from Lab 12

Read the current robot pose  $(x, y, \theta)$  from your existing odometry implementation.

02

## Set a Goal Pose

Define goal =  $(x\_goal, y\_goal)$  in the odometry frame.

03

## Compute and Send $(v, \omega)$

Calculate heading error, set  $(v, \omega) = go\_to\_goal(pose, goal)$ , and send to Turtlebot or unicycle simulator.

04

## Stop at Goal

Robot halts when  $d < 0.25m$ . Verify with multiple goal positions.



LAB 13.3

# Lab 3: Combining Goal + Obstacle Avoidance

## Integration Task

Merge the outputs of Lab 1 and Lab 2 using the `select_action()` priority function:

- Obstacle detected → use avoidance action
- Path clear → use goal-following

## Test & Log

- Run with a person standing in the robot's path
- Log trajectory (x, y) and zone state over time
- Verify robot resumes goal direction after avoidance

 Logging zone state per frame makes debugging much easier — you can replay what the robot "saw" at each decision point.

## LAB 13.4

## Lab 4: Potential Field Simulation

```
def compute_force(pos, goal, obstacles,
                  K_att=1.0, K_rep=0.5, d_max=1.5):
    F = K_att * (goal - pos)
    for obs in obstacles:
        d = np.linalg.norm(pos - obs)
        if d < d_max:
            F += K_rep / d**2 * (pos - obs) / d
    return F
```

Simulate the robot moving through matplotlib animation. **Intentionally create a local minima scenario** — place obstacles so  $F_{att}$  and  $F_{rep}$  cancel — and analyze how to escape it. This builds deep intuition before running real hardware.

## Visualizing the Potential Field Trajectory

```
pos = start.copy(); traj = [pos.copy()]
for _ in range(500):
    F = compute_force(pos, goal, obstacles)
    pos += 0.05 * F / np.linalg.norm(F)
    traj.append(pos.copy())
    if np.linalg.norm(pos - goal) < 0.1:
        break

traj = np.array(traj)
plt.plot(traj[:,0], traj[:,1])
plt.scatter(*goal, color='green', s=100, label='Goal')
plt.show()
```

- ① Normalizing  $F$  before stepping ensures constant step size — the robot moves at uniform speed through the field regardless of force magnitude.

LAB 13.5

# Lab 5: Full Turtlebot Integration Demo

## 1 Connect to Turtlebot via SDK

Establish Bluetooth or WiFi connection and verify telemetry is being received.

## 2 Launch AI Thread + Control Loop

Run obstacle detection in background thread; control loop reads shared state at 20 Hz.

## 3 Test 3 Scenarios

Straight path to goal · Obstacle on the left · Obstacle center-blocking.

## 4 Record and Submit

Film a video demo of all three scenarios. Video is submitted alongside the code.

## Reality Check: What Won't Go as Planned

### YOLO False Positives

Robot avoids objects that don't exist — shadows, reflections, or background clutter trigger detections.

**Fix:** Set confidence threshold  $\geq 0.5$ .  
Confirm detection across 3 consecutive frames before acting.

### Odometry Drift Misaligns Goal

Robot drifts off course over time, making the estimated goal direction increasingly inaccurate.

**Fix:** Recompute goal heading every control cycle from the latest odometry pose, not just at startup.

## Common Errors & Fixes

| Symptom                           | Root Cause                       | Fix                                                |
|-----------------------------------|----------------------------------|----------------------------------------------------|
| Robot spins forever               | Arrival condition never triggers | Check threshold value and pose coordinates         |
| Robot won't steer around obstacle | Confidence or zone check failing | Debug-print zone state every frame                 |
| Robot turns randomly              | YOLO false positives             | Raise confidence threshold, add frame confirmation |
| Jerky, unsmooth motion            | Commands switching abruptly      | Add low-pass filter to $v$ and $\omega$            |

## Smoothing Control Commands

### Exponential Moving Average (EMA)

$$\begin{aligned}v_{\text{smooth}} &= \alpha * v_{\text{cmd}} + (1-\alpha) * v_{\text{prev}} \\ \omega_{\text{smooth}} &= \alpha * \omega_{\text{cmd}} + (1-\alpha) * \omega_{\text{prev}}\end{aligned}$$

Typical value:  $\alpha = 0.3\text{--}0.5$ . Lower  $\alpha$  = smoother but slower response.

### Why Smoothing Matters

- Robot wheels cannot change direction instantaneously
- Reduces mechanical vibration and battery drain
- Transitions between behaviors feel natural rather than snappy
- Improves tracking camera footage for demo video

## Extension: Person Following

### The Idea

Instead of avoiding a person, the robot **tracks and follows** them. The bounding box center becomes a temporary moving goal.

### Control Logic

- Box centered → move straight forward
- Box left of center → turn left
- Box right of center → turn right
- Box too large (person very close) → stop
- No person detected → stop and wait

This is a natural extension for teams who finish the main labs early — it combines Weeks 4, 12, and 13 in a single elegant demo.



## Advanced Demo Scenarios for Week 14

### **Person Following**

Turtlebot follows a walking person through a room. Integrates Weeks 4, 10, 12, and 13.

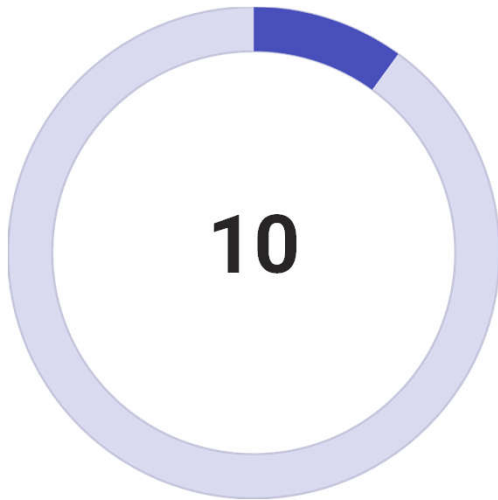
### **Obstacle Course Navigation**

Robot autonomously finds a path through a tabletop environment with multiple static and dynamic obstacles.

### **Marker-Based Goal Arrival**

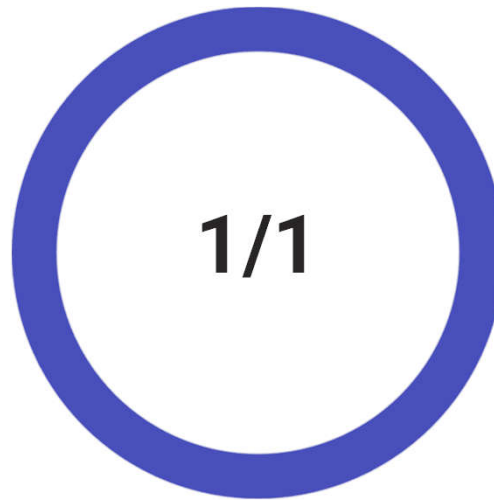
ArUco marker placed at destination. Robot navigates and stops precisely when it visually confirms the marker.

## Evaluating Your Obstacle Avoidance System



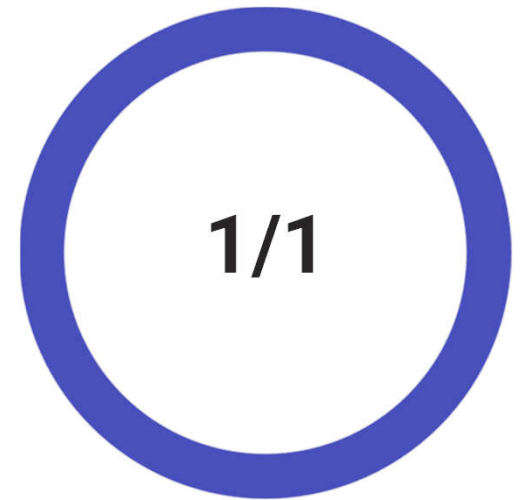
### Success Rate

How many out of 10 runs does the robot reach the goal without collision?



### Trajectory Efficiency

Is the path smooth? Does it take unnecessary detours or oscillate?



### Completion Time

How long vs. a straight-line drive with no avoidance? Aim for  $<2\times$  overhead.

## Repulsion Profile Variants

### Inverse Square — $1/d^2$

Most common. Repulsive force grows very rapidly as the robot approaches an obstacle. Strong avoidance behavior close to obstacles but minimal effect at distance.

$$F_{\text{rep}} = K_{\text{rep}} / d^{**2}$$

### Inverse Distance — $1/d$

Softer profile. Less oscillation in narrow corridors, but the influence zone extends further. Better for wide-open environments.

$$F_{\text{rep}} = K_{\text{rep}} / d$$

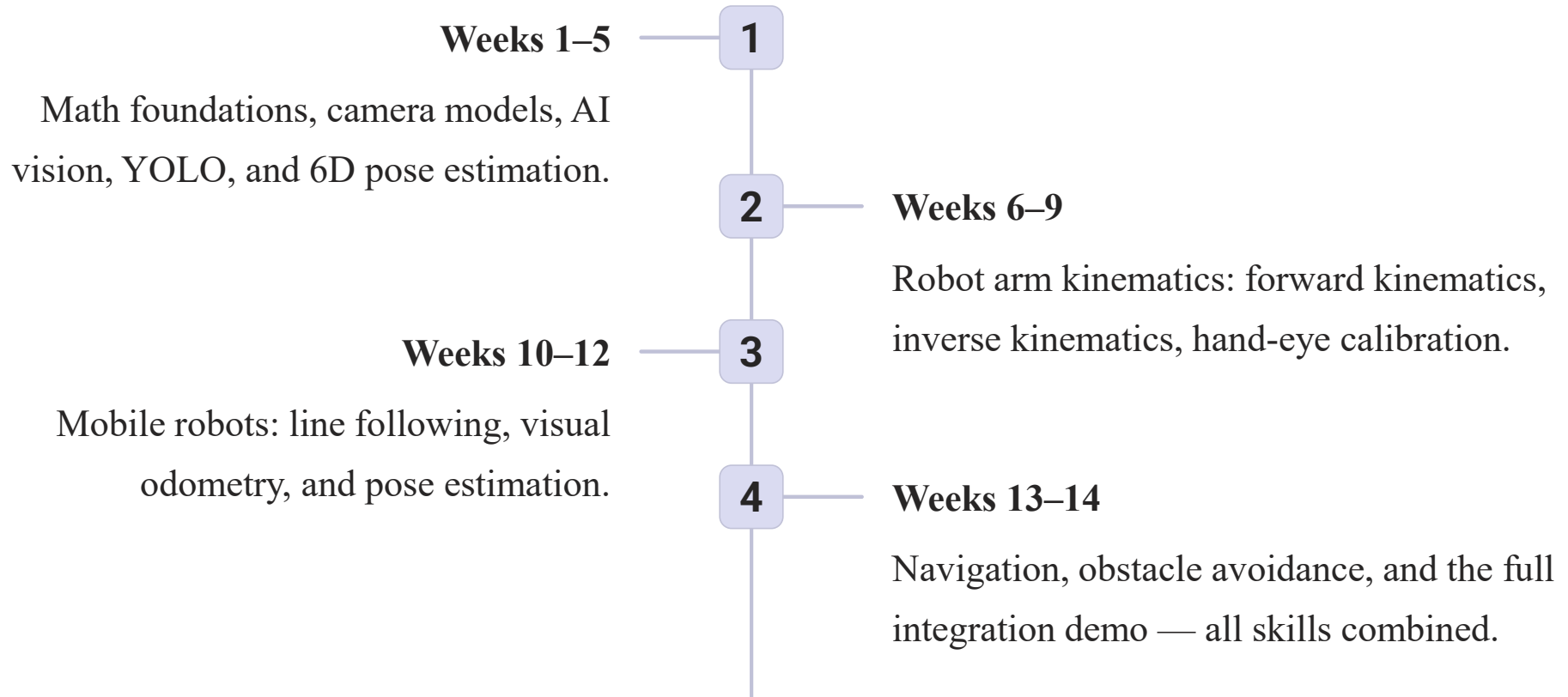
Test both profiles in simulation before running on the real robot — the behavioral differences are striking and build lasting intuition about force design.

## Industry: What Replaces Basic VFH?

| Method                         | Used In                                |
|--------------------------------|----------------------------------------|
| DWA (Dynamic Window Approach)  | ROS Navigation Stack, TurtleBot ROS    |
| TEB (Timed Elastic Band)       | Indoor industrial robots               |
| MPC (Model Predictive Control) | High-end autonomous vehicles           |
| Learned Navigation (RL)        | Boston Dynamics Spot, warehouse robots |

The reactive principles you learn this week are the conceptual foundation for all of these. DWA, in particular, is a direct successor to VFH and is the default in ROS Nav Stack.

## Looking Back: The Full RCV Journey



## Preparing for Week 14

Three deliverables to complete before the final integration sprint:

1

### Stable Code

AI thread + control loop must run for at least **2 minutes** without crashing on real or simulated hardware.

2

### Demo Video

At least **one recorded run** where the robot successfully avoids 2 obstacles and reaches the goal.

3

### Failure Analysis

Document at least **3 failure cases** with explanations of root cause and how you addressed or would address each.

# Week 13 Grading Rubric

| Component           | Weight | Requirement                                                    |
|---------------------|--------|----------------------------------------------------------------|
| Zone classification | 20%    | Correctly classifies all 3 zones, labels shown on image        |
| Goal-following      | 20%    | Robot reaches goal in simulation or real hardware              |
| Obstacle avoidance  | 30%    | At least 2 out of 3 test scenarios succeed                     |
| Code quality        | 15%    | Comments present, functions separated, emergency stop included |
| Result analysis     | 15%    | Strengths, weaknesses, and improvement ideas explained         |

## 5 Core Takeaways from Week 13

### 1 Reactive Navigation

React instantly to sensor data — no full map required. Practical and sufficient for many real scenarios.

### 2 Zone-Based Detection

A simple, effective first step. Surprisingly capable for indoor lab demos despite requiring no depth sensor.

### 3 Potential Field

Elegant explanation, but requires local minima handling. Great for teaching vector-based control intuition.

### 4 Thread Architecture

Separating AI inference from the control loop is the correct pattern for any real-time robotic system.

### 5 Emergency Stop

Non-negotiable when running real hardware. Always implement try/finally with guaranteed wheel stop.



Week 13 Complete 🚀

# Obstacle Avoidance & Autonomous Navigation

The final building block before the integration demo —  
AI · Odometry · Navigation · Control, all working together.

UP NEXT: WEEK 14 — FULL INTEGRATION DEMO

